

Ord1819Resuelto.pdf



Anónimo



Algorítmica



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

Correcto.

¿ES CON V O CON B?
Con C, de Correcto.

¡Pruébalo aquí!



#escribeunfuturomejor

GANAR 2 MALETAS **LLENAS DE** **Kinder** **BUENO**



1

2. (1 punto) Calcule el orden de eficiencia en el caso peor del siguiente código, suponiendo que la llamada a la función V es $O(1)$.

```

void F(T *a, int b, int c) {
    if (b < c) {  $\rightarrow O(1)$ 
        F(a, b, (b+c)/2);
        F(a, (b+c)/2+1, c);
        V(a, b, c);  $\rightarrow O(1)$ 
    }
}
    
```

2. llamada recursiva con tamaño $\frac{n}{2}$

$$F(n) = F\left(\frac{n}{2}\right) + F\left(\frac{n}{2}\right) + 1$$

$$F(n) = 2F\left(\frac{n}{2}\right) + 1$$

Tenemos que hacer un cambio de variable $n = 2^m$:

$$F(2^m) = 2F(2^{m-1}) + 1$$

Separamos parte homogénea de la no homogénea.

$$F(2^m) - 2F(2^{m-1}) = 1$$

Solucionamos la parte homogénea:

$$t_m = x - 2 \rightarrow \text{tenemos como raíz } 2$$

Solucionamos la no homogénea, que es del tipo $b^m \cdot f(m)$,

donde $f(m) = m^0$ y $b = 1$, teniendo como raíz 1 con multiplicidad 0+1

Entonces nuestra ecuación de tiempo queda como $T(n) = c_{11} \cdot 2^m + c_{21} 1^m$

1. SUBE UNA FOTO A @ CON TU KINDER BUENO FAVORITO

2. DINOS CON QUE AMIGO BUENO TE IRÍAS DE VIAJE

3. ¡ETIQUÉTANOS Y DESCUBRE SI HAS GANADO!

DESCUBRE MÁS EN @KINDERBUENO_ES
Promoción válida en España del 1/05/2023 al 30/06/2023 para mayores de 18 años.
Más información y bases legales en nuestro perfil de Instagram: @kinderbueno_es



WUOLAH

Destacamos el cambio de variable: $m = \log_2 n$

$$T(n) = C_{11} \cdot 2^{\log_2 n} + C_{21} = C_{11} \cdot n + C_{21} \in O(n)$$

3. (1.5 puntos) Diseñe un método eficiente para multiplicar enteros largos.

Suponemos que cualquier número entero con n dígitos es expresado de la siguiente forma:

$$A = a_{n-1}a_{n-2}a_{n-3} \dots a_1a_0, \text{ siendo } a_i \text{ el dígito } i.$$

El número entonces sería el siguiente:

$$A = a_{n-1} \cdot 10^{n-1} + a_{n-2} \cdot 10^{n-2} + \dots + a_1 \cdot 10 + a_0$$

Así podríamos dividir el problema en subproblemas, dividiendo el problema en 2, dividiendo los números de la siguiente manera:

$$A = a_1 \cdot 10^{n/2} + a_0$$

$$B = b_1 \cdot 10^{n/2} + b_0$$

$$A \cdot B = (a_1 \cdot 10^{n/2} + a_0)(b_1 \cdot 10^{n/2} + b_0) =$$

$$= a_1 \cdot b_1 \cdot 10^n + a_1 \cdot b_0 \cdot 10^{n/2} + a_0 \cdot b_1 \cdot 10^{n/2} + a_0 \cdot b_0$$

Dividiendo así el problema en 3 subproblemas:

$$A \cdot B \begin{cases} a_1 \cdot b_1 = S_1 \\ (a_0 + a_1) \cdot (b_0 + b_1) = S_2 \\ a_0 \cdot b_0 = S_3 \end{cases}$$

Nos faltará la combinación de los 3 subproblemas,

de cual lo haremos de la siguiente manera

$$S_1 \cdot 10^n + (S_2 - S_1 - S_3) \cdot 10^{n/2} + S_3 = A \cdot B$$

#YoElijo Cerveza SIN

Sea cual sea
el vehículo que
conduces, elige
cerveza SIN.

WWW.CONDUCCIONRESPONSABLECERVEZASIN.COM



UNA GRAN CERVEZA.
UNA GRAN RESPONSABILIDAD.

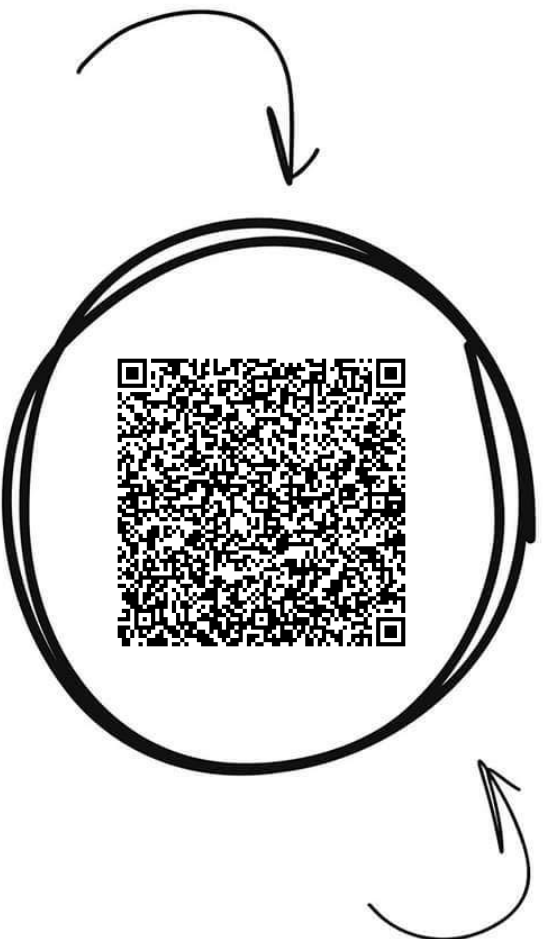
© CONDUCCIÓN RESPONSABLE, CERVEZA SIN es una iniciativa de la Asociación de Cervecerías de España con el apoyo de la Dirección General de Tráfico.



Algorítmica



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la UGR

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR

Para calcular la eficiencia de veces en el pseudo algoritmo:

Función $S = \text{multiplica}(A, B)$

Si $n=1$ entonces

$$S = A \cdot B$$

En otro caso, hacer:

Dividimos A en (a_1, a_0) y B en (b_1, b_0)

$$S_3 = \text{multiplica}(a_0, b_0)$$

$$S_2 = \text{multiplica}(a_0 + a_1, b_0 + b_1)$$

$$S_3 = \text{multiplica}(a_1, b_1)$$

$$S = S_3 \cdot 10^n + (S_2 - S_3 - S_1) \cdot 10^{n/2} + S_3$$

Fin en otro caso

return S ;

Es decir, la función de recursión sería la siguiente:

$$T(n) = 3T(n/2) + n$$

Hacemos cambio de variable $n = 2^m$

$$T(2^m) = 3T(2^{m-1}) + 2^m$$

$$T(2^m) - 3T(2^{m-1}) = 2^m$$

$p(x) = x - 3 \rightarrow$ tenemos una raíz 3, por lo que

La parte no homogénea sería $2^m - m^0$, por lo que tenemos de raíz 2

$$\begin{aligned} t_m &= C_1 \cdot 3^m + C_2 \cdot 2^m \xrightarrow{m \rightarrow \log_2 n} C_1 \cdot 3^{\log_2 n} + C_2 \cdot 2^{\log_2 n} = \\ &= C_1 \cdot 3^{\log_2 n} + C_2 \cdot n = C_1 \cdot n^{\log_2 3} + C_2 \cdot n \in \boxed{O(n^{\log_2 3})} \end{aligned}$$

4. (1.5 puntos) Explique un problema de caminos mínimos y resuélvalo con Programación Dinámica.

El problema es el siguiente:

Dada una matriz de adyacencia, busquemos hallar la distancia mínima entre cada par de nodos.

Para ello haremos uso de programación dinámica, pues busquemos minimizar el problema.

Para ello, haremos de una table de $n \times n$, donde n es el número de nodos que tenemos.

La casilla $M[i][j]$ represente el cost de llegar desde i hasta j con un valor mínimo.



HAZTE UN BOWL
A TU BOWL



5

Para el algoritmo, comprobaremos el valor mínimo de llegar hasta un nodo j , pasando por u o directamente desde el nodo i .

Tenemos el siguiente caso base:

- Si $i = j$, el coste será 0.

El algoritmo será el siguiente

Algoritmo Distancia Mínima (Matriz Adyacencia)

M = Matriz Adyacencia.

para $u=0$ hasta n

para $i=0$ hasta n

para $j=0$ hasta n

coste Directo = $M[i][j]$

coste Intermedio = $M[i][u] + M[u][j]$

$M[i][j] = \min(\text{coste Directo}, \text{coste Intermedio})$

WUOLAH

Ejemplo de traza.

Para $n = 4$, tenemos la siguiente matriz de Adyacencia:

$$G = \begin{pmatrix} 0 & 4 & \infty & 7 \\ 20 & 0 & 18 & \infty \\ \infty & 12 & 0 & 7 \\ 17 & \infty & \infty & 0 \end{pmatrix}$$

Empezaremos comprobando si pasando por el nodo 0 tiene mejor costo

$$G = \begin{pmatrix} 0 & 4 & \infty & 1 \\ 20 & 0 & 18 & 21 \\ \infty & 12 & 0 & 7 \\ 17 & 21 & \infty & 0 \end{pmatrix}$$

desde 1 hasta 0 y desde 0 hasta 3
 $3 \xrightarrow{18} 0$ y $0 \xrightarrow{4} 1$

Comprobamos pasando por 1

$$G = \begin{pmatrix} 0 & 4 & 22 & 1 \\ 20 & 0 & 18 & 21 \\ 32 & 12 & 0 & 7 \\ 17 & 21 & 39 & 0 \end{pmatrix}$$

$0 \xrightarrow{4} 1$ y $1 \xrightarrow{18} 2$
 $2 \xrightarrow{12} 1$ y $1 \xrightarrow{20} 0$
 $3 \xrightarrow{21} 1$ y $1 \xrightarrow{18} 2$

Pasando por 2

$$G = \begin{pmatrix} 0 & 4 & 22 & 1 \\ 20 & 0 & 18 & 21 \\ 32 & 12 & 0 & 7 \\ 17 & 21 & 39 & 0 \end{pmatrix}$$

Pasando por 3

$$G = \begin{pmatrix} 0 & 4 & 22 & 1 \\ 20 & 0 & 18 & 21 \\ \textcircled{24} & 12 & 0 & 7 \\ 17 & 21 & 39 & \text{''} \end{pmatrix}$$

+ $2 \xrightarrow{7} 3$ y $3 \xrightarrow{17} 1 = 24 < 32$

Siendo G la matriz con distancias mínimas

5. (2 puntos) Sea un laberinto codificado en una matriz C de tamaño m filas y n columnas. Una celda $C(i,j)=1$ si la celda es transitable, y $C(i,j)=0$ si no lo es. La casilla de salida del laberinto también es transitable y tiene valor $C(i,j)=2$. Desde una casilla (i,j) transitable se podría viajar a cualquier casilla $(i+1, j)$, $(i-1, j)$, $(i, j+1)$, $(i, j-1)$, siempre que sean transitables. Diseña un algoritmo con la técnica BackTracking que, dada una casilla (x,y) inicial, permita encontrar un camino para salir del laberinto desde dicha casilla.

Para solucionar el problema usaremos una técnica de Backtracking.

Tenemos:

- Representación: Un nodo representa la posición x,y .
- Función de poda: Si $C[i][j]=0$, lo generamos pues no es transitable.
- Árbol: en cada nivel, nos movemos una casilla.

Cada acción será sumar o restar 1 a la i y lo mismo con la j , es decir, movemos arriba, abajo, derecha o izquierda. El nodo raíz sería la posición (x,y) .

- Restricción implícita: el valor de i no puede ser mayor que m y j no puede ser mayor que n .

- Restricción explícita: no podemos sumar más de 1 a una sola coordenada ni sumar a ambas coordenadas. (No sé si es correcto).
- Función solución: si $C[i][j] = 2$, terminamos.

Para el algoritmo, dado que buscamos un camino, cada nodo tiene su padre, para así poder devolver el camino final.

El procedimiento es el siguiente:

Tenemos una pila P , donde meteremos los nodos vivos

Inicialmente P tiene el nodo raíz

$Vértice = P.pop()$

Mientras P no está vacío:

Para cada hijo procedente de $Vértice$

Función Solución $\rightarrow$ Si hijo es una solución, la devolvemos

Función Poda $\rightarrow$ En otro caso, si no es igual a 0.
Añadimos el hijo a P .

Fin Para

Asignamos a $Vértice$ $P.pop()$.

Fin Mientras



HAZTE UN BOWL
A TU BOWL



9

De esta manera, si la solución es mala, no hay una salida desde x, y .

Este algoritmo tiene un factor de ramificación de 4, y un factor de profundidad de $m \cdot n$, donde en el peor caso no hay ninguna casilla que no sea accesible y no se pueda nada; por lo que el orden sería de $O((m \cdot n)^4)$ en el peor de los casos.

1. (1 punto) Defina el concepto de algoritmo y describa sus propiedades. Exponga un ejemplo de *pseudo*-algoritmo que incumpla alguna de las propiedades, explicando por qué no es un algoritmo.

Ver en diapositivas de teoría.

WUOLAH